Programmable In-Network Timing Obfuscation for DNP3

Anonymous Submission

***Abstract*—An adversary watching a power-grid control network can tell which device it is looking at from how long that device takes to answer, because vendors implement the same function differently. The relay cannot be changed to hide this, and existing traffic obfuscation does not help: it reshapes flow-level sizes and inter-packet times, while the feature that identifies the device is one interval inside a single transaction, and it hides its added packets behind encryption that these plaintext networks do not have.**

We put the defense in the network. A programmable switch in front of the outstation holds the device's acknowledgment and its response, and releases each at a deadline computed from a single anchor. Neither release depends on how long the device took, so the interval an observer measures is a value the operator sets. The switch forwards the packets it received, so both endpoints run unmodified and no DNP3 byte changes.

We implement this on an Intel Tofino-1 and measure it against a physical SEL-751A relay. The device's signature leaves the interval: it settles on the configured value with a spread more than two orders of magnitude smaller, so measuring again gains an adversary almost nothing, and a classifier trained beforehand is left guessing. An adversary who retrains recovers much of it from a second interval we do not target, which is the bound on what this achieves.

***Index Terms*—traffic obfuscation, device fingerprinting, DNP3, programmable switches.**

I. Introduction

Device fingerprinting has been an essential step in cyber reconnaissance, allowing adversaries to reveal unique features of target networks and to design effective, stealthy attack strategies. These techniques are becoming increasingly critical in industrial control systems (ICSs) such as power grids, where adversaries often use IP-based control networks and computing devices within as entry points to inflict physical damage. Consequently, fingerprinting shifts the focus from visited websites and user biometric behavior to device models and types of control operations that are critical to ICS attacks. In the 2015 attack that disrupted Ukrainian power grids and the Stuxnet attack that disrupted Iranian nuclear power facilities, it is widely believed that adversaries stay in their systems for at least 6 months to perform cyber reconnaissance [1], [2].

To disrupt device fingerprinting, many studies present network traffic obfuscation [3], [4]. Device fingerprinting targeting general computing environments generally relies on network-level features such as packet size and/or inter-packet latency observed from communication patterns [5]. Consequently, existing traffic obfuscation often focuses on (i) padding and splitting network packets, which hide or change the distribution of network packet sizes [3], and (ii) delaying network packets and adding dummy ones, which disrupt the inter-packet timing pattern [4], [6]. Since manipulating communication networks can introduce runtime overhead, recent works have begun to offload traffic obfuscation onto programmable network switches, which change communication patterns at much higher line rates than CPUs [7]–[9].

Unfortunately, these methods cannot be directly applied to ICS environments for two main reasons. First, device fingerprinting methods on ICS devices rely on different features from the ones in general computing environments. Instead of using network layer knowledge, they attempt to use behavior on physical devices to reveal device model or types of operations. For example, work in [10] uses the latency between the TCP acknowledge packets and the actual response from the same device to estimate execution time there. Since each vendors may choose different materials or algorithms to implement the same control functionality, this time can be accurate in fingerprinting those devices. Second, existing traffic obfuscation is performed over encrypted channels, which are necessary to mix the obfuscated and normal traffic. Unfortunately, ICS networks still rely on unencrypted traffic due to a wide range of legacy devices, despite ICS network protocols may provide security features. Directly padding or adding dummy packets can easily reveal obfuscated packets, exposing the the underlying traffic pattern to adversaries.

In this paper, we present a framework that schedules when selected packets of an ICS transaction leave the network, so that a timing feature an observer measures becomes a value we choose rather than a property of the device. A programmable switch at the outstation edge holds the acknowledgment and the response of a transaction and releases each at a deadline it computes when the transaction begins. Because the switch forwards the same packets it received, the endpoints run unmodified firmware and every DNP3 byte reaches the master unchanged, which is what the plaintext setting requires. The framework is general over which packets are held and what the released interval should be. We demonstrate it on the two timing features that Formby et al. [10] use, the cross-layer response time of a poll and the master-visible timing of a control command, and we evaluate the first of these in depth. These are two case studies of one mechanism, not two systems.

Network and Distributed System Security (NDSS) Symposium 2027
22–26 March 2027, Seoul, Republic of Korea
ISBN 978-1-970672-09-1
https://dx.doi.org/10.14722/ndss.2027.23xxxxx [DRAFT: paper number not assigned]
www.ndss-symposium.org

This paper makes the following contributions:

- **Design a timing-obfuscation framework** that replaces a selected within-transaction timing feature with a configured value by scheduling the release of real protocol packets, and derive the coupling between the acknowledgment hold, the response hold, and the target interval that any such schedule must satisfy (Section IV).
- **Realize the framework on an Intel Tofino-1** using paired blocker and hold queues that sustain a hold until a deadline expires, with an acknowledgment-anchored read lane, a request-anchored control lane, and forwarding that stays open when a response arrives after its scheduled release (Section V).
- **Evaluate the realization against a physical SEL-751A relay** and report how closely the released interval reaches its target, what the policy can and cannot cover, the latency the mechanism adds, the hardware resources it uses, and what an adaptive attacker still learns from the features that remain (Section VI).

II. Background and Motivation

A defense that changes when a packet leaves has to be built on the events an observer can actually see. This section names those events for the two DNP3 transactions the paper covers, explains why the interval between two of them tells an observer something about the device, states which of the events we measured and which we did not, and describes what a programmable switch makes possible.

The DNP3 read and control transactions. DNP3 carries supervisory control and data acquisition traffic over TCP between a master and an outstation [11]. The master polls the outstation with a READ request, and the outstation returns the requested measurements in an application-layer response. A control command instead uses select-before-operate: the master sends a SELECT that names an output point, the outstation returns a SELECT response, the master then sends an OPERATE, and the outstation confirms it with a solicited OPERATE response. Because DNP3 runs over TCP, the outstation's transport layer also acknowledges each request before its application answers. Each transaction on the wire is therefore a request, a transport acknowledgment carrying no DNP3 payload, and a DNP3 response, in that order, as Figure 1 shows. We use those three words for those three packets throughout the paper.

Why the interval between them is a fingerprint. The two answers come from different parts of the outstation. Its TCP stack emits the acknowledgment as soon as the request arrives, whereas its application emits the response only after the device has done the work the request asks for. The interval between them, the cross-layer response time or CLRT, therefore measures the outstation's own processing and not the network path, and it differs between device models and between the kinds of work a device is asked to do. Formby et al. used exactly this interval to tell device types and models apart in a substation network [10]. Our own measurements show the same effect on a single device: with no timing

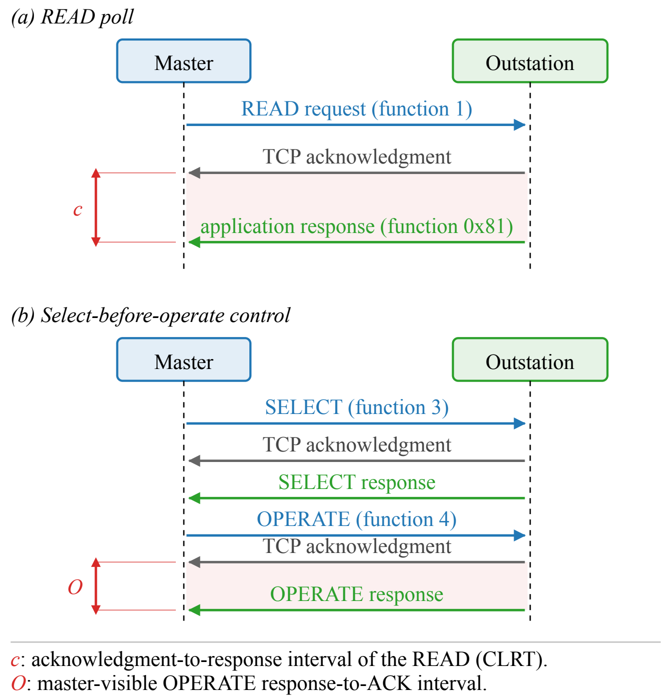

Fig. 1. The DNP3 transactions the paper measures, as seen on the master-facing link; requests in blue, transport-layer acknowledgments in grey, and application responses in green. (a) A READ poll: the request, the outstation's transport-layer acknowledgment, and its application-layer response. The interval c between the acknowledgment and the response is the cross-layer response time (CLRT). (b) A select-before-operate control: SELECT and OPERATE each produce an acknowledgment and a solicited application response. The interval O between the OPERATE's acknowledgment and its solicited response is the master-visible OPERATE response-to-ACK interval.

defense the median CLRT on an SEL-751A is 2.116 ms for a READ, 2.050 ms for a SELECT and 2.937 ms for an OPERATE, each with a long tail, so the three classes of transaction are already partly separable from timing alone (Section VI). The DNP3 function codes travel in plaintext and already name each operation, so the timing is not what reveals that an operation happened. What it adds is a second, device-derived signature of that operation, one that needs no payload and would remain visible if the payload were encrypted.

What we observe, and what we do not. Every quantity the paper measures is a packet timestamp taken on the master-facing link: the request, the acknowledgment, and the response. The instants at which the switch releases a packet, and every event on the relay-facing link, were not captured; where the paper names them, they come from the configuration and we say so. No measurement in this paper observes physical breaker motion.

What a programmable switch makes possible. A programmable switch exposes a match-action pipeline, written in P4 [12], that parses headers, keeps state in registers, and places packets into output queues at line rate. The property that matters here is that such a pipeline can delay a packet and release it later without a host in the loop. A switch placed between the master and the outstation can therefore hold the

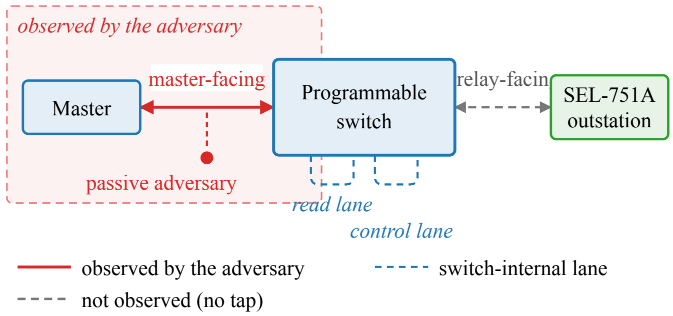

Fig. 2. Observation model. The master reaches the SEL-751A outstation through one programmable switch. The passive adversary sees only the master-facing link (solid red, shaded region); the relay-facing link and the two switch-internal scheduling lanes (dashed) are not observed, by the adversary or by our measurements.

acknowledgment and the response of a transaction and release them on a schedule an operator sets, changing neither endpoint nor any byte of the payload. That possibility is what the rest of the paper develops; the next section states precisely which observer it is meant to defeat, and what we require of it.

III. Threat Model and Objectives

The adversary. We assume a passive observer on the master-facing link, the adversary of the fingerprinting attack we defend against [10]. Figure 2 shows where it sits. It records TCP and IP headers, segment sizes and arrival times. It does not modify or inject packets, compromise the master or the outstation, control the switch, or read the switch's internal state. It does not see the relay-facing link, which in our deployment is inside the switch with no tap on it, and it does not see physical breaker motion. Because DNP3 is commonly deployed without transport security on shared substation networks [13], we assume the traffic is unencrypted, so the observer also reads the DNP3 function codes directly. An adversary that probes, injects, or otherwise acts on the network is out of scope; the passive observer is the setting the traffic-obfuscation work we build on also targets.

What the adversary is looking for. Reconnaissance precedes the attacks that matter here: false data injection, for instance, requires the attacker to know which devices it is dealing with [14]. Fingerprinting supplies that knowledge, and it does not need the payload, because the timing of the exchange carries it. Following Formby et al. [10], two intervals are in scope. The first is the CLRT of a READ or of the SELECT phase of a control, which measures the outstation's own processing as Section II describes. The second is the master-visible interval between an OPERATE's acknowledgment and its solicited response. The adversary also sees the request-to-acknowledgment interval, which the mechanism does not target and which we report as measured.

The second interval deserves a careful statement, because it is easy to claim more for it than we can support. Formby et al. estimate the time a device physically takes to complete an operation, but they do so from a sequence-of-events timestamp carried inside the outstation's application payload; their alternative estimate from packet arrival times produced no usable result [10]. The interval we measure is a packet-timing feature of the solicited control response. It is not physical actuation time, we do not measure physical actuation time anywhere, and we make no claim about the payload-timestamp feature, which a mechanism that moves packets without editing bytes could not affect in any case.

What our evaluation actually separates. Device fingerprinting is the application that motivates the work. What we evaluate is narrower, and we keep the two apart throughout. Our testbed has one outstation, so there is no second device to confuse it with and no device identification to measure. What we measure is whether the timing of an exchange still reveals *which class of transaction* it was, on that one device. A result here is therefore the suppression of a class-conditioned timing feature, and not a demonstration that two devices become indistinguishable.

Two adversaries, not one. A defense evaluated only against a classifier retrained on its own output answers the wrong question, so we separate two cases. The *fixed* adversary learns transaction classes from undefended traffic and applies that model unchanged once the defense is deployed; this is the adversary the reconnaissance setting describes, because the profile is built before the defense is met. The *adaptive* adversary collects obfuscated traffic and retrains on it. That is a strictly stronger assumption than the setting requires, and it bounds what remains learnable. We report both, and the second is the one that limits our claim.

Research objectives. We state the goal as five properties, each of which the evaluation answers in turn.

- **RO1 (read timing).** The visible CLRT of READ and of the SELECT phase of SBO is a configurable policy value rather than the outstation's own processing signature.
- **RO2 (control timing).** The master-visible OPERATE response-to-ACK interval stays at a policy value while the relay-facing hold is drawn from a configured codebook.
- **RO3 (timing leakage).** The timing features carry less information about the transaction class, and a fixed adversary that trained on undefended traffic is reduced to approximately three-class chance.
- **RO4 (release policy and coverage).** The release budget, the range of policies the mechanism can realize, the fraction of traffic it can cover, and the residual it leaves are characterized rather than assumed.
- **RO5 (cost and protocol preservation).** The latency the mechanism costs and the frames and bytes it adds are measured, and the external DNP3 workload still completes: every request is answered and every control operation returns a success status.

What is out of scope. Three boundaries follow from the model above and hold for every result we report. The framework does not hide the DNP3 function codes, which stay in plaintext, so RO3 concerns the timing channel and not the command semantics. It does not change packet sizes, and this paper makes no size-related claim. And the relay-facing link is not instrumented, so the per-transaction hold and the release toward the outstation are configured rather than observed, which is why RO2 is stated in terms of the master-visible interval alone.

RO1 and RO2 both ask for an interval the operator chooses. Whether a switch can produce such an interval at all, and what it costs to do so, is a question about timing rather than about adversaries, and the next section answers it.

IV. Design

The objectives of Section III require a selected interval to become a value the operator sets. This section explains how that is possible at all. It first states a timing model for one transaction, which is enough to see what has to change and why one arrangement of holds works where another does not. It then asks which quantities are actually free to choose and what limits each of them. It closes with the control transaction, whose anchor has to differ, and with what the design leaves untouched. The hardware realization is deferred to Section V: a reader should be able to follow this section without knowing anything about P4.

A. A timing model for one transaction

Consider one READ transaction, drawn in Figure 3. The master sends a request. The outstation answers it twice: first its transport layer acknowledges the request, and later its application layer returns the response with the data. We write t_A for the instant the acknowledgment reaches the switch and t_R for the instant the response reaches it. The switch forwards each packet onward at some later instant, e_A and e_R respectively. The master therefore observes the interval $e_R - e_A$, up to the propagation and capture delay of the two packets on the link between them.

The interval that carries information about the device is

$$\mathrm{CLRT}_{\mathrm{original}} = t_R - t_A, \tag{1}$$

the time the outstation itself spends between acknowledging the request and producing the answer. Section II explains why this interval identifies the device; here it is simply the quantity we intend to remove.

The switch changes what the master sees by holding each packet. We call the two holds

$$D_A = e_A - t_A, \qquad D_R = e_R - t_R, \tag{2}$$

the *acknowledgment hold* and the *response hold*. Substituting the two definitions into the interval the master observes gives the identity that governs the whole design,

$$\mathrm{CLRT}_{\mathrm{new}} = e_R - e_A = \mathrm{CLRT}_{\mathrm{original}} + D_R - D_A. \tag{3}$$

Equation (3) makes the difference between two kinds of defense concrete. If the two holds are constants, their difference is a constant, and the observed interval is the original interval moved by that constant. The distribution slides along the axis and keeps its width, so an observer who has seen the device before can still recognize it by the shape. To remove the device's own contribution the holds cannot both be constants: D_R must absorb the variation in $\mathrm{CLRT}_{\mathrm{original}}$.

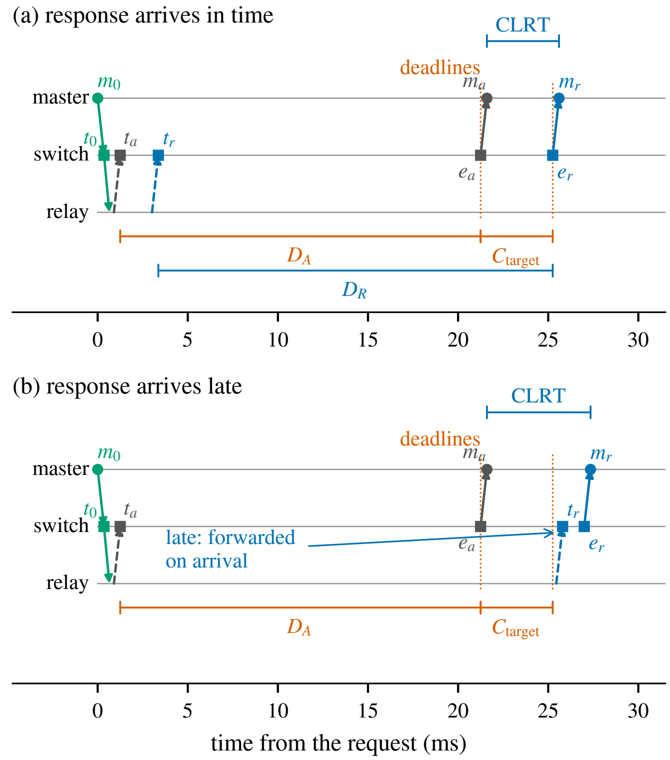

Fig. 3. Release timeline of one READ transaction. The switch arms both deadlines from a single anchor, the arrival of the outstation's acknowledgment at t_a: the acknowledgment is due at $t_a + D_A$ and the response at $t_a + D_A + C_{\mathrm{target}}$, marked by the dotted lines. In (a) the response arrives before its deadline, so the master observes the configured target. In (b) it arrives after, and the switch forwards it on arrival. The response hold D_R is drawn in (a) to show that it is not configured: it is whatever the schedule requires, so it moves with the arrival t_r. Only m_0, m_a and m_r were measured; the four switch-side instants were not. Propagation and the switch's processing term are drawn at a legible size, not to scale.

Our mechanism does this by scheduling both releases from one anchor. When the acknowledgment arrives, the switch computes both release instants immediately,

$$e_A = t_A + D_A, \qquad e_R = t_A + D_A + \mathrm{CLRT}_{\mathrm{target}}, \tag{4}$$

where $\mathrm{CLRT}_{\mathrm{target}}$ is the interval we want the master to observe. Because both instants derive from t_A and neither derives from t_R, the term $\mathrm{CLRT}_{\mathrm{original}}$ cancels in (3) and the observed interval is the configured one. The response hold that results is not chosen but implied,

$$D_R = D_A + \mathrm{CLRT}_{\mathrm{target}} - \mathrm{CLRT}_{\mathrm{original}}. \tag{5}$$

Equation (5) is the reason the three quantities cannot be picked independently. Once the acknowledgment hold and the target are fixed, the response hold follows from whatever the device happened to do on that transaction, and it varies from one transaction to the next exactly as much as the device does.

This holds only while the response exists in time to be released on schedule. The switch can delay a packet but cannot emit one it has not received, so (4) requires

$$t_R \leq t_A + D_A + \mathrm{CLRT}_{\mathrm{target}}. \tag{6}$$

Note that the acknowledgment may leave before the response arrives; that ordering is normal and does not by itself imply a lost packet. When (6) fails, the deadline is already in the past when the response appears and the switch forwards it immediately. The master then observes an interval larger than the target, as in Figure 3(b). We report these transactions rather than excluding them, because they are the boundary of the mechanism and not a defect of the measurement.

B. Choosing the holds and the target

The model leaves three quantities to decide: the acknowledgment hold D_A, the target $\mathrm{CLRT}_{\mathrm{target}}$, and, through (5), the response hold D_R that follows from them. Each is limited by a different constraint.

The acknowledgment hold is limited in principle by the transport. Holding an acknowledgment delays the feedback the master's TCP uses to decide that a segment is lost, so a hold that approached the master's retransmission timeout would make the master resend a request the outstation had already answered. Two things make this bound less tight than it first appears. The timer is adaptive: RFC 6298 [15] computes it from the round-trip times the stack has itself observed, so a hold that is applied uniformly is folded into those samples and raises the timeout rather than racing it. The standard notes the same effect from the attacker's side, as a way to inflate a sender's timer [15, §6]. What that argument does not cover is a hold that varies between transactions, which enters the variance term instead. Our policy holds by a constant, which is the case the adaptation absorbs.

In this deployment the transport bound is not the one that binds. The mechanism sustains a hold with a finite resource, described in Section V, and cannot hold a packet past a horizon H that the control plane computes from that resource; for the configuration we evaluate, H is 30.8 ms. Subtracting a conservative bound on the outstation's own acknowledgment latency, the measured detection and release terms, and a safety margin leaves an admissible budget of 24.8 ms, which the control plane enforces before it installs a policy. That ceiling is an order of magnitude below any retransmission timeout the master could plausibly use, so the mechanism reaches its own limit long before the transport notices. We report the margin we observed in Section VI-D and we do not claim the tested hold is universally safe: the master's realized timer was never read back, and it is the one measurement that would settle the transport question directly.

The target is limited by the operation, and here we could not find a standard that fixes it. We looked for a communications requirement that applies to a DNP3 poll or a select-before-operate control on a distribution relay, and the only numeric deadline we could verify from primary documentation is the relay's own select-to-operate timeout, whose default is one second. Timing classes defined for other protocols govern those protocols' own messages and do not transfer to a DNP3 poll merely because the numbers are convenient. We therefore treat the target as a deployment choice rather than a compliance question, state the value we tested, and report the latency the mechanism adds so that an operator with a requirement of their own can check it against that number.

The acknowledgment hold and the target together set what the mechanism costs. Their sum

$$D = D_A + \mathrm{CLRT}_{\mathrm{target}} \tag{7}$$

is the release budget: by (6) it decides which transactions the switch can place on the schedule at all. A larger budget covers more of the device's own response-time distribution and costs more delay on every transaction it covers. Note that D is the budget, not the delay added to each exchange: a transaction the outstation answers quickly is held longer than one it answers slowly, and the added delay is at most D rather than equal to it.

Because the budget and the observable are different quantities, an operator can fix the cost and still choose the feature. Holding D at 24 ms, we installed targets from 1 ms to 22 ms and the master observed each one; the delay the mechanism added stayed the same across all of them. Section VI-D reports that sweep and the point at which the realized hold stops tracking the requested one.

The framework and the policy it carries are separate. The model above says nothing about what $\mathrm{CLRT}_{\mathrm{target}}$ should be; a constant target is the objective we chose, because a constant removes the device's variation entirely and is the simplest policy to state and to check. A schedule could instead draw the target from a distribution, or make it a function of the operation, and the same mechanism would carry it. We implement and evaluate the constant target, and we do not claim evidence for the others.

C. The control transaction needs a different anchor

A select-before-operate control command differs in one way that matters here: the framework holds the command itself before it reaches the relay, so that the moment the master issued the command is separated from the moment the relay acts on it. The outstation's acknowledgment then cannot be the anchor, because that acknowledgment does not exist until after the command has been released, and any deadline measured from it would carry the length of the command hold into the interval the master observes.

The control lane is therefore anchored to the request. When the OPERATE arrives at T_0, the switch releases it toward the relay after an internal delay J, and places the acknowledgment and the solicited response the master sees at configured offsets from T_0,

$$e_{\mathrm{ack}} = T_0 + A, \qquad e_{\mathrm{or}} = T_0 + R. \tag{8}$$

The interval the master observes is then

$$O = e_{\mathrm{or}} - e_{\mathrm{ack}} = R - A, \tag{9}$$

in which J does not appear. Consequently the relay-facing release time can vary while the master-facing interval does not, and an observer cannot subtract a known offset to recover it. The control plane checks that A exceeds the largest admissible J plus the outstation's own acknowledgment latency before it installs the values, since a deadline the switch cannot honor would fail open rather than protect. In our experiments $R - A$ and $\mathrm{CLRT}_{\mathrm{target}}$ are both 4 ms, so the two lanes present the same value to the observer.

D. What the design does not change

The mechanism replaces one interval per lane and leaves the rest of the exchange as it was. The DNP3 function codes remain in plaintext and the direction of every packet remains visible, so an observer can still tell a poll from a control command by reading the packet. The request-to-acknowledgment interval still contains the outstation's own acknowledgment latency, displaced by the constant D_A but not reshaped by it, so whatever that interval reveals it continues to reveal. The switch forwards the packets it received without padding them, injecting new ones, or editing any DNP3 field, which is what the plaintext setting requires and which also means the mechanism changes no size or ordering feature. Section VI-F reports what an attacker who is allowed to retrain on the obfuscated traffic still recovers from the features that remain.

V. Implementation

Section IV says when each packet should leave. A forwarding pipeline has no way to wait: a P4 program runs to completion per packet, can read a clock, but cannot sleep or schedule a callback. This section explains how the switch releases a packet at a chosen instant anyway.

A. Holding a packet with queues

What the switch does have is a traffic manager with several queues per port and a strict-priority scheduler, and a scheduler is a device for deciding what does *not* leave. Each lane owns two queues on one internal port, as Figure 4 shows. The *hold queue* carries the packet we delay. The *blocker queue* sits above it in strict priority and carries small packets the switch generates itself. While blockers keep arriving, the held packet is never served. The hold ends when the blocker queue runs dry. Blockers circulate on an internal loopback port and never reach a cable, so neither endpoint can observe them, and each carries a tag naming its transaction and lane so a leftover blocker cannot delay a later transaction.

It would be natural to read this as a timer built from a count: if a blocker takes τ to drain, K blockers hold a packet for about $K\tau$. The program does not measure time that way. When the anchoring packet arrives it stores an absolute release instant, quantized to a 256 ns grid, and every recirculating blocker compares the current timestamp against it. While the difference is negative the blocker is regenerated; once it is not, the queue empties and the held packet is served. The deadline decides when the packet leaves; the blockers only have to

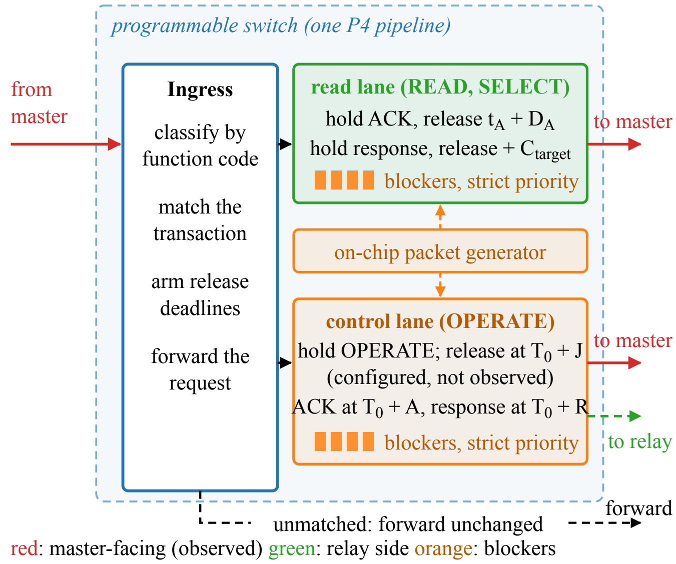

Fig. 4. How one pipeline realizes the schedule. Ingress classifies each DNP3 transaction and arms its release deadlines. The read lane holds the acknowledgment and the response of a READ or SELECT and releases them relative to the outstation's acknowledgment; the control lane holds an OPERATE, releases it toward the relay at a configured delay that we do not observe, and releases the acknowledgment and solicited response relative to the request. The packet generator fills the blocker reservoirs (orange), and a strict-priority scheduler drains them before the held packet, which is what sustains a hold until its deadline expires. Red arrows are master-facing and observed; the green dashed arrow is the configured relay-facing release, which is not. Unmatched traffic is forwarded unchanged.

keep the queue non-empty until then, so the reservoir is sized for continuity rather than for duration. This is what makes the coupling of Section IV-A implementable: the response's release instant is computed at the acknowledgment, so the resulting hold absorbs whatever the outstation does without the switch ever measuring it.

B. One READ transaction

The request arrives, a table classifies it by function code, and the program forwards it at once, recording that a transaction is in flight and which acknowledgment to expect. When the acknowledgment arrives at t_A the program writes both release instants from that one timestamp, and does so once from an unarmed sentinel so a duplicate acknowledgment cannot arm a second deadline. The acknowledgment enters the hold queue and the packet generator fills the blocker queue. A response that arrives before its deadline queues behind the acknowledgment, and the two leave in order at their scheduled instants. A response that arrives after it has nothing to wait for, so the program forwards it at once, as in Figure 3(b). When the response has left, the transaction state is cleared.

Read and control responses can be in flight together, so each treatment has its own loopback port, its own hold and blocker queue pair, and its own registers. Sharing one hold queue would let whichever waited longer delay the other and corrupt the policy the second was meant to present.

C. Failing open

Every path through the program ends in a forwarding decision. A packet outside the protected session, an unrecognized request, an acknowledgment with no armed transaction, and a response whose transaction has retired are all forwarded unchanged: such an exchange loses its protection but not its delivery. Every hold is bounded by a budget on blocker passes, which releases the packet and clears the state if the reservoir has not drained within the horizon H of Section IV-B. With the timing mode off the same program forwards everything immediately, which is how we collected the Timing OFF arm. The program holds and forwards packets and does nothing else to them, so every byte the master reassembles is the byte the relay produced.

D. Resources and testbed

The program is about 1,560 lines of $P4_{16}$ for the Tofino Native Architecture, compiled with the Barefoot SDE 9.13. It occupies all twelve ingress match-action stages of one pipeline and six egress stages, with 112 tables. State is small: one-entry registers reached by stateful ALU actions, one access per packet per register. One 25 Gb/s port carries the master and one 1 Gb/s port the relay; two further ports are loopbacks carrying no cable and are the two lanes.

Every policy quantity is a runtime table entry, so an operator changes policy by writing entries rather than loading a program. The loaded build realizes the dual-deadline schedule only: its arming table admits that mode alone, so the acknowledgment-only and response-only limits of the model are properties of the design and not settings of this build. Configuring them left both packets unheld, and the measured interval was the outstation's own.

The control plane, about 890 lines of Python, brings up ports and queues, installs the packet-generator configuration, writes the timing parameters and reads them back. It also refuses configurations the data plane could not honor, checking the admissible budget of Section IV-B and, on the control lane, that A exceeds the largest admissible J plus a bound on the outstation's acknowledgment latency. The outstation is an SEL-751A feeder protection relay; the master issues READ and select-before-operate operations to isolated control points through a guarded driver, so no control operation in the evaluation moves a breaker.

VI. Evaluation

In this section, we evaluate the framework against the five objectives of Section III, RO1 to RO5, on the physical testbed of Section V.

A. What we measured, and where

Evaluation Cases. We compare two arms of the same program. In *Timing OFF*, the switch forwards the outstation's acknowledgment and response as they arrive; in *Obfuscated*, it holds them and releases them on the deadlines of Section IV. The two arms differ only in the release schedule. The switch is the only path between the master and the relay, and, because all timestamps are taken on the master-facing link, every interval reported in this section is one that a passive observer on the control network can record.

Concretely, the interval we report is the time between the arrival of the transport acknowledgment and the arrival of the application response, both observed on the master's own interface. The switch's internal instants, when the outstation's packets reach it and when it releases them, are on links that were not captured, so they are recovered from the program rather than measured.

Dataset. We report per DNP3 exchange, i.e., one request, the transport acknowledgment that follows it, and the application response. The campaign is 22 grouped collection runs in one approximately five-hour window, with six captures of 480 exchanges per run and 132 captures in total. It contains 63,360 exchanges, 31,680 per arm, of which 26,400 are READ, 2,640 SELECT, and 2,640 OPERATE in each arm. Every request was answered, with no retransmission and no malformed frame, and every capture holds 1,448 frames and 130,708 bytes. To characterize the release policy itself, we collected a separate sweep on the same testbed, 5,860 further exchanges over 19 captures: 16 configured release policies of the dual-deadline mode the loaded program realizes, and three control captures taken with the timing mechanism disabled.

B. Does the released interval reach its target? (RO1)

Effectiveness in RO1. We achieve RO1 if the read-path schedule of (4) replaces the outstation's own CLRT with the policy value. We compare the CLRT of READ and of the SELECT phase of SBO between the two arms over all 22 grouped runs as full empirical distributions, because the shape of the Timing OFF distribution is what can make a single observation informative to an adversary.

Figure 6 shows the two READ distributions directly. Before obfuscation the interval is broad and multi-modal, spanning 1.01 to 83.46 ms with distinct peaks near 1.2, 2.1 and 4.0 ms; after it, 99.9% of transactions fall within 0.10 ms of the configured target. The sample variance falls from 6.808 to 0.394 ms^2, a reduction of 94.2%, and the sample standard deviation from 2.609 to 0.628 ms. The zoom panels show what the full-range view cannot: the released interval is not a single value but a narrow distribution about the target, and the Timing OFF arm places only 13.5% of its transactions anywhere in that window.

In Figure 5(d), we show the median CLRT under Timing OFF is 2.116 ms for READ and 2.050 ms for SELECT, with interquartile ranges of 2.778 ms and 2.697 ms. Under the mechanism, both medians are 4.000 ms and the interquartile range of each falls to 0.006 ms, a reduction of more than two orders of magnitude in spread. Figure 5(a, b) shows why the median alone understates the change: the Timing OFF distributions rise in a small number of discrete steps, so a single observation can be informative about the class, whereas the obfuscated distributions are a step at the scheduled release. This is because the release instant is computed from the outstation's acknowledgment and a fixed offset, and thus,

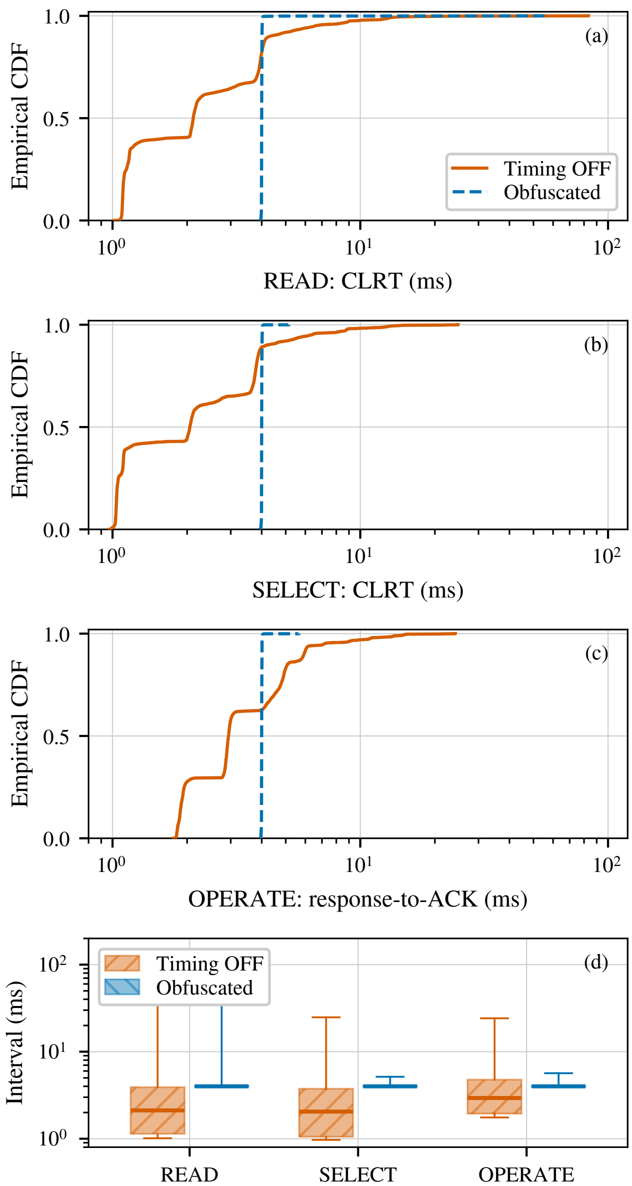

Fig. 5. **Measured interval per transaction class, over 22 grouped runs.** (a) READ and (b) the SELECT phase of SBO report the cross-layer response time; (c) OPERATE reports the master-visible response-to-ACK interval, which is a different anchor and not a complete SBO transaction. The abscissa is logarithmic and spans the full support, so no observation is clipped. (d) The same measurements as quartiles, with whiskers spanning the full support.

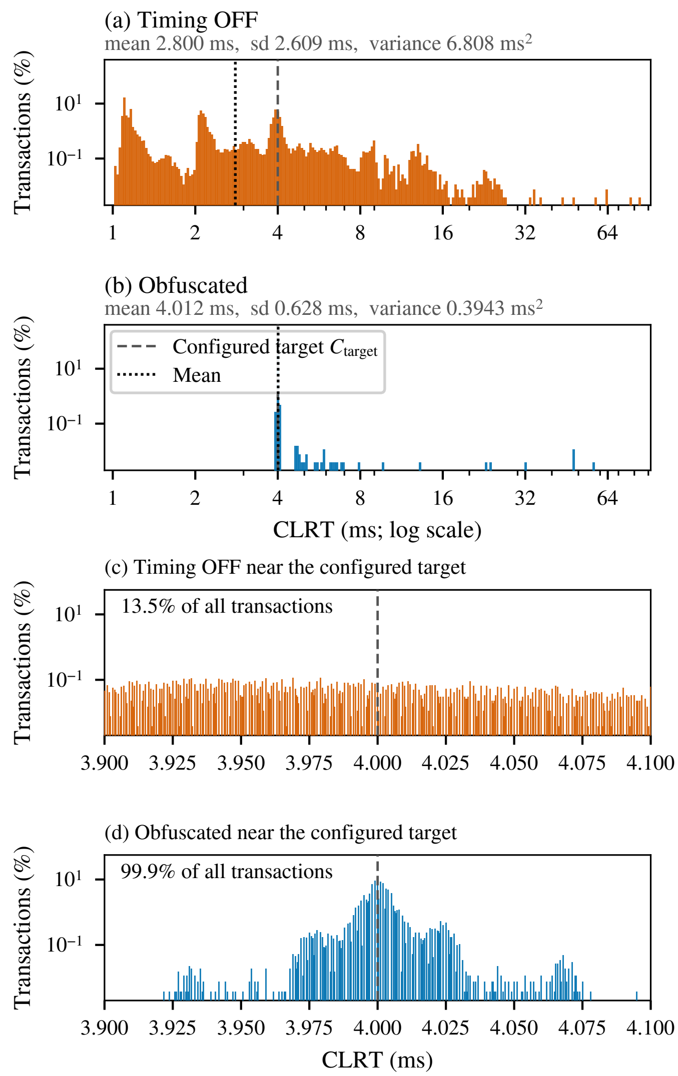

Fig. 6. **Measured READ CLRT before and after obfuscation.** (a) and (b) show the full measured range on logarithmic axes, on common bins and identical limits. (c) and (d) show a linear window of 0.10 ms either side of the configured target. Bars are the percentage of that arm's own transactions in each bin, and in (c) and (d) the denominator is still the arm's full sample, so the share falling inside the window is a share of everything measured. The dashed line is the configured target, a policy value; the dotted line is the measured mean. Nothing is smoothed and no tail is clipped.

it no longer depends on how long the device took. The change is a replacement, not a shift: the standard deviation falls from 2.609 to 0.628 ms for READ and from 2.267 to 0.024 ms for SELECT, variance ratios of 0.058 (95% CI [0.019, 0.101]) and 0.0001 (95% CI [1.4×10^{-5}, 3.4×10^{-4}]) under a session-level bootstrap. A defense that merely shifted the interval would keep the Timing OFF spread, because translating a sample leaves its variance unchanged: the counterfactual of the Timing OFF distribution translated onto the protected median retains a standard deviation of 2.609 and 2.267 ms, one to four orders of magnitude above what we measure. That counterfactual is analytical, computed from the Timing OFF samples themselves. The loaded program has no shifting mode, so it is not a measured arm and we do not present it as one. The spread collapses because both releases share the acknowledgment anchor, as Section IV sets out. A shift keeps the shape and moves it; the measurement concentrates instead. Low spread at the wrong interval would not be normalization, so we also report where the distribution sits: both medians are the configured 4.000 ms, and the interquartile range of 0.006 ms is two orders of magnitude below the 1 ms step between neighbouring policy settings in the sweep of Section VI-D. A tail remains, and it is one-sided: the largest obfuscated READ interval is 57.182 ms, and 24 of 26,400 READ exchanges depart from the scheduled release by more than 1 ms, while none falls materially below it. The asymmetry is the mechanism, not the data. A response that reaches the switch after its scheduled release cannot be moved backwards, so it is forwarded on arrival and lands above the target; nothing can place one early. These are the coverage boundary of RO4 seen from the other side.

Stability within the Campaign. In Figure 7, we plot each grouped run in acquisition order. The obfuscated median sits at the scheduled release in every run and for every class, while the Timing OFF medians stay separated. Consequently, the effect is not an artifact of one run or of drift over the five hours.

C. Does the control lane behave the same way? (RO2)

Effectiveness in RO2. We achieve RO2 if the control-path rule of (8) holds the master-visible response-to-ACK interval at the policy value. Because the control path is anchored to the request, its observable is $O = R - A$, in which the internal hold J does not appear, and the read-path budget D does not apply to it. The switch draws J per transaction from the configured codebook $\{2, 6, 12\}$ ms. As shown in Figure 5(c, d), the OPERATE median moves from 2.937 ms under Timing OFF to 4.000 ms under the mechanism, with the interquartile range falling from 2.827 ms to 0.006 ms, and the master-visible interval remained concentrated near the configured 4 ms policy value across all 22 grouped runs. This is a statement about the interval the master sees; the relay-facing release at $T_0 + J$ is not observed (Section VI-G).

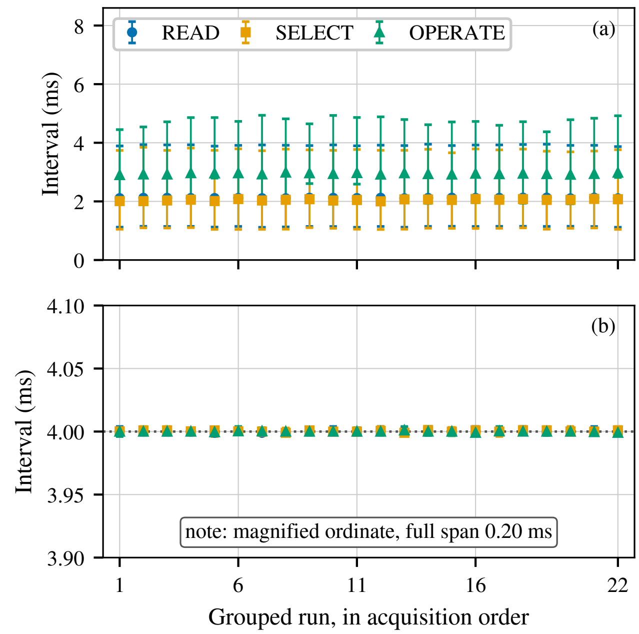

Fig. 7. **Within-campaign stability across the 22 grouped runs**, in acquisition order. (a) Timing OFF and (b) Obfuscated. Markers are the run median and bars span the interquartile range. *Panel (b) uses a magnified ordinate spanning only 0.20 ms around the scheduled release.* The 22 runs come from one approximately five-hour campaign on one relay behind one switch and are not independent replications.

D. What range of settings does the mechanism support? (RO4)

Effectiveness in RO4. We achieve RO4 if the interval a passive observer records is a policy value the operator sets, over a usable range of settings. To measure that range on the switch itself, we installed 16 release policies in turn and ran a fixed DNP3 workload through each. Two series answer the question. The first holds the total budget at $D = D_A + \mathrm{CLRT}_{\mathrm{target}} = 24$ ms and moves the split between the two, which changes the visible CLRT while leaving the total unchanged. The second fixes the target at 4 ms and raises D_A until the mechanism leaves its operating region. Each point is the median over its READ exchanges.

In Figure 8(b), we show that the measured CLRT follows the configured target along the identity line across the whole series: targets of 1, 2, 4, 8, 12, 16, 20 and 22 ms produce measured medians of 0.998, 1.999, 3.999, 8.001, 12.001, 16.000, 20.001 and 22.001 ms, while the end-to-end response time stays between 25.30 and 25.34 ms at every point. As a result, the leaking interval and the cost of the exchange can be set independently, which is what makes the mechanism a framework rather than one fixed setting. In Figure 8(c), the request-to-acknowledgment interval tracks D_A up to 30 ms, where the measured median is 30.571 ms and the CLRT still sits at 4.001 ms. Beyond that point, the acknowledgment interval saturates at about 31.07 ms and the CLRT can no longer be held: it rises to 5.503, 7.498 and 9.507 ms at D_A of 32, 34 and 36 ms. The reason is that the reservoir that starves

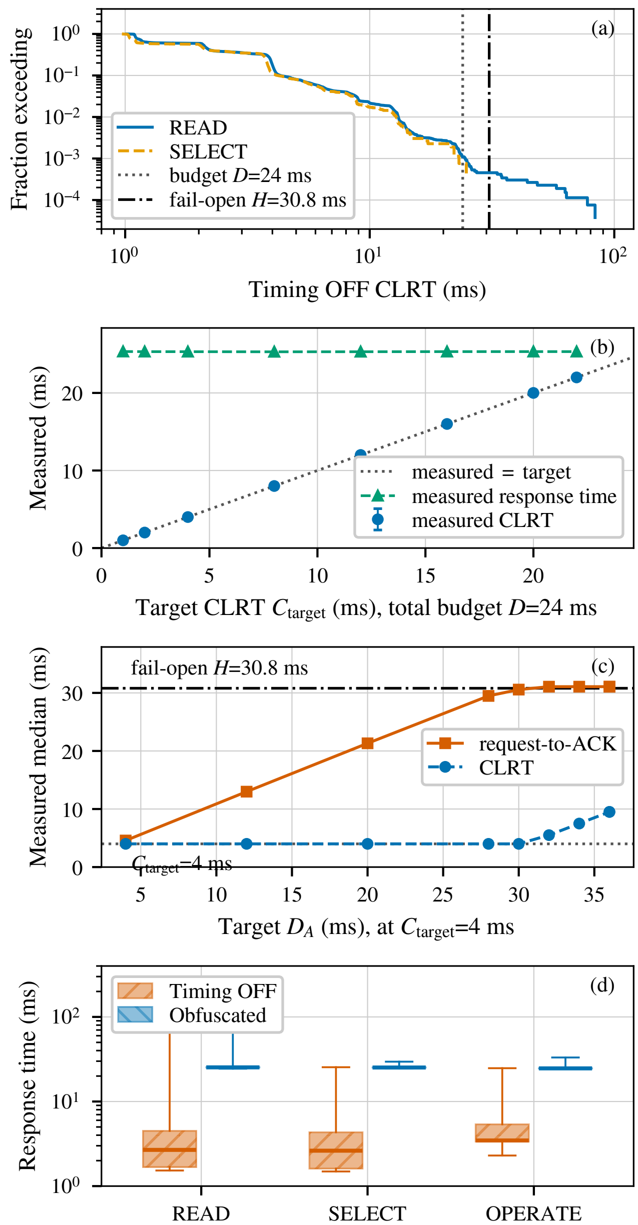

Fig. 8. **The release policy is programmable, and its budget is bounded on both sides.** (a) Fraction of Timing OFF read-lane exchanges, READ and the SELECT phase of SBO only, whose CLRT exceeds a given value, with the release budget D and the fail-open horizon H. (b) Measured hardware sweep at a fixed total budget $D = 24$ ms: the visible CLRT follows the configured target along the identity line while the end-to-end response time stays put. (c) Measured sweep of D_A at a fixed target: the request-to-acknowledgment interval tracks the target until it saturates near H, beyond which the CLRT can no longer be held at its target. (d) End-to-end response time per class. Bars in (b) span the interquartile range; boxes in (d) span the quartiles with whiskers over the full support.

the queue carries a finite pass budget, so a hold cannot outlast it, and the saturation sits close to the fail-open horizon H of Section IV. From below, the range is closed by the outstation's own response time, which the switch cannot anticipate: in Figure 8(a), the budget of 24 ms covers 99.900% of Timing OFF read-lane exchanges, and 29 of 29,040 arrive too late to be held, 28 of them READ and one SELECT.

E. What does the mechanism cost? (RO5)

Effectiveness in RO5. We achieve RO5 if the latency the mechanism costs and the frames and bytes it adds are measured, and the external DNP3 workload still completes. As shown in Figure 8(d), the median response time rises by 22.7 ms for READ, 22.6 ms for SELECT and 21.2 ms for OPERATE. The added latency is close to, but below, the release budget, because it depends on how long the outstation itself took; it is a real operational cost that an operator would weigh against the polling period and any protocol timeout. On the master-facing link, each capture carries 1,448 frames and 130,708 captured bytes for 480 exchanges in both arms, i.e., 3.017 frames and 272.3 bytes per exchange, identical in the two arms. In other words, the mechanism moves packets in time without adding a frame or a byte to that link. Across all 63,360 exchanges, every request was answered with a well-formed response, and all 5,280 SELECT and OPERATE exchanges returned a success status, so the DNP3 workload completes unchanged under the mechanism.

Timeout and Retransmission Headroom. Holding the acknowledgment consumes the master's timers, so we measured what it consumed. On the master-facing captures we recovered every transport-level event across all 63,360 exchanges: the request bytes were acknowledged in a separate segment every time, never piggybacked on the response, and no request was sent while earlier bytes were still unacknowledged. There was no retransmission, no reset and no duplicate acknowledgment in either arm. The longest the master waited for its request to be acknowledged was 29.2 ms and the longest it waited for the response was 77.7 ms. We did not record the master's kernel configuration, so its actual retransmission timeout is unknown; for scale, the minimum Linux documents is 200 ms and RFC 6298 recommends a 1 s floor, and no retransmission occurred. The receive timeout in our driver is 3 s, and it bounds one read rather than the whole transaction, so it is not a transaction deadline; we report the measured completion times against it without asserting that the two coincide, which would need application-level receive tracing we did not collect. Select validity is the other timer a held control transaction could violate, since the master issues its OPERATE only 0.19 ms after the SELECT response reaches it and the mechanism delays that response: none of the 2,640 obfuscated OPERATE exchanges was refused for a stale select. All of this characterizes this operating point on this testbed, and would have to be rechecked for a master with a shorter timeout.

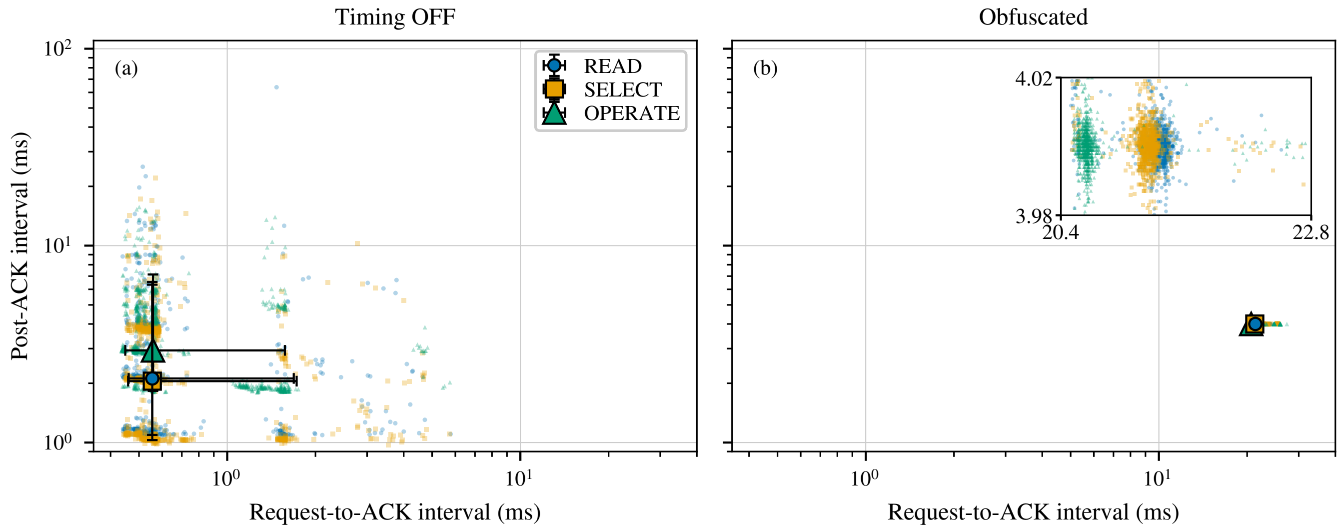

Fig. 9. **Targeted timing-feature collapse and residual anchor leakage.** The two intervals a passive observer can measure, plotted against each other on identical logarithmic axes: (a) Timing OFF and (b) Obfuscated. The ordinate is the post-acknowledgment interval, the cross-layer response time for READ and the SELECT phase of SBO and the master-visible response-to-ACK interval for OPERATE. Small marks are a deterministic, class-stratified subsample drawn for legibility; the large markers are the median and the bars the 5th to 95th percentile, both computed on the complete dataset. Under the mechanism the vertical, device-derived interval of all three classes collapses onto the policy value and READ and SELECT overlap, while OPERATE keeps a horizontal offset because the control lane is anchored to the request and the read lane to the outstation's acknowledgment. The inset magnifies the collapsed band on linear axes. This figure shows timing-feature overlap among transaction classes on one physical outstation. It is not clustering performance, not device identification, and not evidence that different devices become indistinguishable.

F. *What does the attacker still learn? (RO3)*

Setup. We achieve RO3 if the timing features carry less information about the transaction class and a fixed adversary trained on undefended traffic is reduced to chance. We treat this as a three-class problem over READ, SELECT and OPERATE, for which chance balanced accuracy is one third, with the two independent intervals, request to acknowledgment and acknowledgment to response, as features; the total response time is their sum and is excluded. We estimate mutual information for the CLRT in bits and compare it against a null built by shuffling class labels within each grouped run over 1,000 permutations, which preserves the per-run class counts. Every fold leaves out one grouped run, and we report the spread of the 22 held-out-run scores as a range.

Effectiveness in RO3. In Figure 9, we show what the mechanism does to the two measurable intervals before any classifier is applied. Under Timing OFF, the three classes are separated mainly along the post-acknowledgment axis, with median intervals of 2.116, 2.050 and 2.937 ms for READ, SELECT and OPERATE against request-to-acknowledgment medians of 0.555, 0.555 and 0.558 ms, which barely separate the classes at all. Under the mechanism, that axis collapses: all three medians sit at 4.000 ms, and READ and SELECT overlap. What remains is a horizontal offset, with the request-to-acknowledgment median moving to 21.336, 21.239 and 20.650 ms. This is because the read lane is anchored to the outstation's acknowledgment and the control lane to the request.

In Figure 10, we show the two attacker models. The fixed adversary trained on Timing OFF traffic reaches 0.651 balanced accuracy on held-out Timing OFF exchanges using the CLRT alone, and 0.733 using both intervals. Applied unchanged to obfuscated traffic, it falls to 0.333 and 0.334, which is chance. Mutual information between the CLRT and the class falls from 0.383 bits to 0.004 bits; the Timing OFF value lies far above its permutation null at the resolution of the test ($p = 0.001$ with 1,000 permutations), and the obfuscated value lies inside its null ($p = 0.096$). For the evaluated fixed Random-Forest attacker, then, three-class transaction identification falls to approximately chance, and the cross-layer response time stops carrying information about the class. The adaptive adversary, retrained on obfuscated traffic, remains at chance on the CLRT alone but recovers to 0.651 when the acknowledgment latency is added. The reason is that the read and control paths are anchored differently, so their acknowledgment latencies differ, and an adversary willing to collect obfuscated traffic and retrain can exploit that difference; the confusion panels show its shape, with OPERATE separable while SELECT stays confused with READ. Closing this residual requires the two paths to share an anchor, which we leave to future work.

G. *Limitations*

One Device, One Campaign. The evidence is one SEL-751A behind one Tofino-1, in one approximately five-hour campaign of 22 grouped runs. Consequently, it does not establish behavior across days, devices, or deployments, and

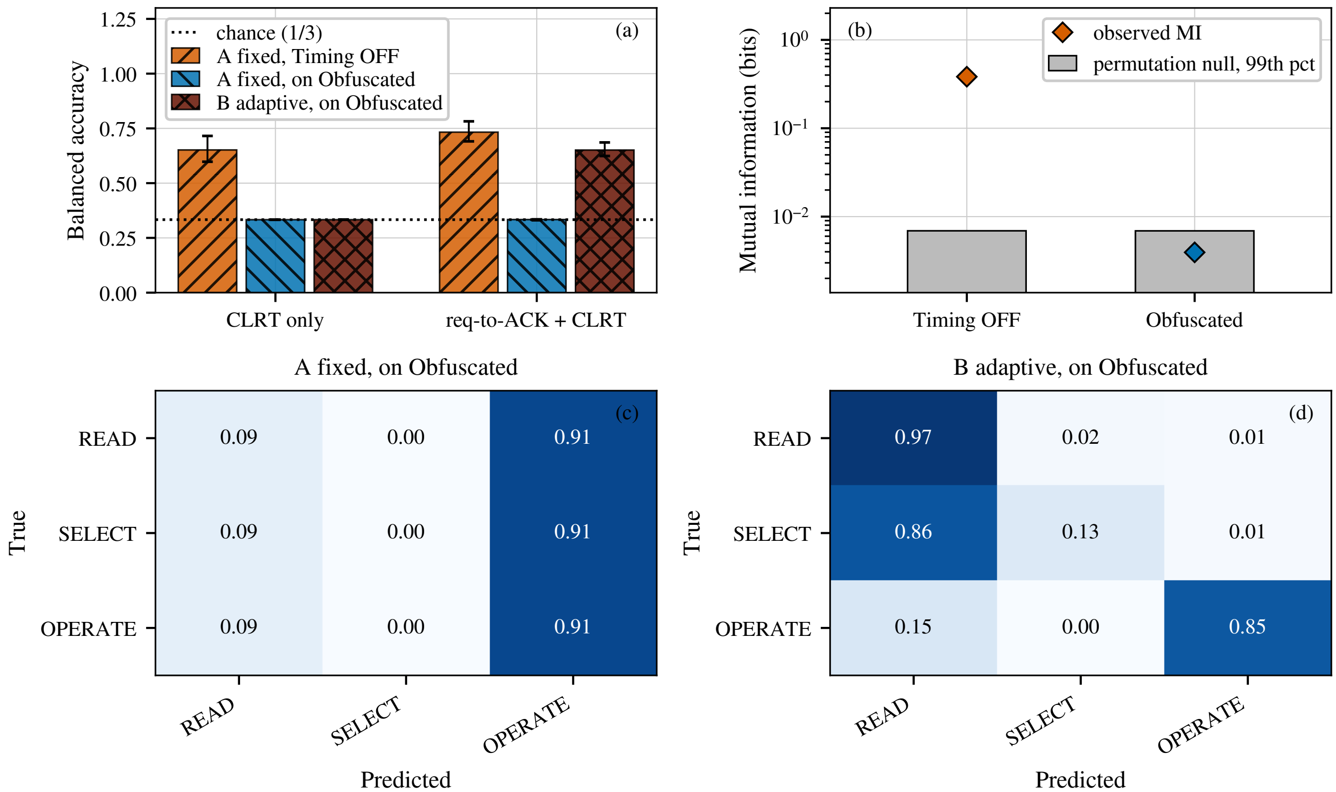

Fig. 10. **Transaction-class leakage under the two attacker models**, leaving out one grouped run at a time over all 22 runs. (a) Balanced accuracy of the evaluated Random-Forest attacker; bars are the mean over the 22 held-out runs and the whiskers span the full range across those runs, which is within-campaign variability and not a confidence interval, because the folds share training data. (b) Observed mutual information between the CLRT and the transaction class against the 99th percentile of a within-run permutation null. (c) and (d) Row-normalized confusion over all 22 held-out runs, using both intervals.

the runs are grouped collections rather than independent replications.

What the Classifier Shows. DNP3 function codes remain visible in plaintext, so the result concerns the timing channel and not the concealment of operations. The classification we evaluate separates transaction classes on one outstation; it is not device-model identification, and with one outstation nothing here shows that two devices become indistinguishable. The attacker results characterize the Random-Forest attacker we evaluated rather than every classifier, and an adaptive adversary recovers part of the class signal from the difference between the two anchors.

What Was Not Observed. Only the master-facing link was captured. Consequently, the realized per-transaction hold J, the relay-facing release at $T_0 + J$, and any physical actuation are configured rather than observed, we report no result per J, and nothing here shows that exactly one OPERATE reached the relay. The fail-open horizon H is computed in the control plane rather than enforced by the data plane, so the saturation of RO4 is an observed transition and not a proof that the two coincide. The sweep offsets come from the configuration table rather than from a per-point readback. The switch's own release instants were likewise not captured, and the loaded program cannot supply them: the timestamp registers it declares are never written, and every write it does declare would take an ingress timestamp rather than an egress one, so obtaining a release instant would require a modified program. And because the program suppresses a retransmission of the response that matches an already-seen transport position, a retransmission by the outstation of a held response would be absorbed inside the switch and would not appear in a master-facing capture, so we cannot exclude one.

Scope. A fixed read-path release budget leaves a measurable late tail rather than eliminating it. The overhead of RO5 counts the master-facing link; the switch's internal blocker traffic never appears there. Size obfuscation is outside this paper: the mechanism changes no packet size, and we make no size, padding, splitting or segmentation claim.

VII. Related Work

Device fingerprinting. Devices can be identified from what they emit, even without any access to the payload. Kohno et al. fingerprint a remote host from the skew of its clock as seen in TCP timestamps [16], and GTID fingerprints a device and its type from the timing signature of its packets [17]. These works define the threat we address; our objective, on the other hand, is to remove the timing features they rely on from the master-facing observation. Their target is the device type, whereas the classifier we evaluate separates transaction classes on one outstation, so we suppress a feature their cross-layer response-time method uses without demonstrating that two devices become indistinguishable.

ICS and power-system fingerprinting. Formby et al. brought fingerprinting to control systems with the cross-layer response time and the physical operation time, measured at the device, of DNP3 outstations [10], and Gu et al. added device physics as a feature [18]. Jeon et al. fingerprint SCADA devices passively without deep packet inspection [19]. The reconnaissance these techniques serve enables false-data injection and process-control attacks [14], [20]. Defenses in this setting have so far worked at the content and connectivity layer: DefRec virtualizes physical functions to present deceptive content [21], RAINCOAT randomizes data acquisition and injects decoy measurements [22], and a companion testbed evaluates such anti-reconnaissance approaches on grid infrastructure [23]. Compared to these, our framework works below the semantics they reshape and complements them: it changes when the wire carries the outstation's answer, not what the answer says.

General traffic obfuscation. Traffic morphing transforms one class's packet-size distribution into another's [3], while Dyer et al. show that per-packet padding and fragmentation still leave exploitable features [4]. Tamaraw fixes packet size and rate at a bandwidth cost [6], and WTF-PAD pads inter-packet gaps adaptively [24]. However, deep fingerprinting attacks defeat several of these [5], and their accuracy at Internet scale is contested [25]. The same size-and-timing channel identifies smart-home devices under encryption, where shaping is the countermeasure [26], and Pacer and NetShaper shape a tenant's traffic to a schedule that bounds side-channel leakage in the cloud [27], [28]. These defenses assume an encrypted payload and a cooperating end host or hypervisor that can add cover traffic. Our framework, in contrast, targets a plaintext protocol whose feature is one interval inside a request-response exchange, adds no byte, and runs where the outstation cannot be changed.

Programmable data-plane traffic transformation. P4 defines the programmable parser and match-action model we use [12]. PIFO defines programmable scheduling at line rate [29], and SP-PIFO approximates it with strict-priority queues on today's switches [30], which is the primitive our reservoirs use. Ditto pads and injects chaff to a fixed pattern at line rate [7], Minos morphs and schedules traffic on a programmable switch at low cost [8], and Securitas combines fragmentation and insertion, and realizes them across several data planes [9]. NetHide obfuscates topology in the data plane against link-flooding reconnaissance [31]. Closest to our setting, Hu et al. use programmable switches to enhance industrial-protocol security [32]. Our framework differs from these in the observable it addresses, i.e., the master-visible timing of a DNP3 transaction, and in holding packets rather than adding or reshaping them.

DNP3 timing and security. East et al. catalogue attacks on DNP3 and the absence of encryption that makes its semantics visible [13], and specification-based and state-based intrusion detection has been adapted to DNP3 [33], [34]. These do not change the timing an observer sees. On the control path we report a different quantity from theirs: the master-visible OPERATE response-to-acknowledgment interval, which is a protocol-response timing feature. Formby et al. estimate physical operation time from a sequence-of-events-recorder timestamp carried in the outstation's application payload, and report that the alternative estimate from unsolicited-response arrival times produced no usable results [10]. A mechanism that moves packets in time and changes no byte cannot alter such a payload timestamp, and the traffic we evaluate carries neither unsolicited responses nor any time-bearing object. We therefore make no claim about physical operation time, and we do not present our control-path result as a correlate of theirs.

VIII. Conclusion

How long an outstation takes to answer tells an observer which device it is, and the relay leaking it cannot be modified. We moved the defense into the network: a switch in front of the outstation holds the packets carrying the device's timing and releases each at a deadline computed when the transaction begins. Both deadlines come from one anchor, so the interval an observer sees is the operator's value rather than the device's.

Measured against a physical relay, the device's signature leaves that interval, and a classifier trained on it beforehand is left guessing. The framework protects the exchanges its budget can schedule and forwards the rest untouched. An adversary who retrains recovers much of what it lost from a second interval we do not target; closing that needs both paths on one anchor. The evidence is one relay, one campaign, one classifier, and we never captured the switch's own release instants, so the residual around the target is a difference between two releases and not a delay we can place inside the switch.

References

[1] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the Cyber Attack on the Ukrainian Power Grid," Electricity Information Sharing and Analysis Center (E-ISAC) and SANS Institute, Defense Use Case, Mar. 2016.

[2] R. Langner, "Stuxnet: Dissecting a Cyberwarfare Weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[3] C. V. Wright, S. E. Coull, and F. Monrose, "Traffic Morphing: An Efficient Defense Against Statistical Traffic Analysis," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2009.

[4] K. P. Dyer, S. E. Coull, T. Ristenpart, and T. Shrimpton, "Peek-a-Boo, I Still See You: Why Efficient Traffic Analysis Countermeasures Fail," in *2012 IEEE Symposium on Security and Privacy (S&P)*, 2012.

[5] P. Sirinam, M. Imani, M. Juarez, and M. Wright, "Deep Fingerprinting: Undermining Website Fingerprinting Defenses with Deep Learning," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

[6] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A Systematic Approach to Developing and Evaluating Website Fingerprinting Defenses," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 227–238.

[7] R. Meier, V. Lenders, and L. Vanbever, "Ditto: WAN Traffic Obfuscation at Line Rate," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

[8] Z. Wang, Q. Li, G. Xie, D. Zhao, K. Li, Z. Fan, L. Ma, and Y. Jiang, "Minos: A Lightweight and Dynamic Defense against Traffic Analysis in Programmable Data Planes," in *Proceedings of the 2025 USENIX Annual Technical Conference (ATC)*, 2025.

[9] G. Xie, Q. Li, Z. Shi, G. Antichi, Y. Zhu, K. Li, C. Weng, S. Miano, Y. Jiang, and M. Xu, "Defending against Traffic Analysis Attacks with Flexible In-Network Obfuscation," in *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026, pp. 2043–2063.

[10] D. Formby, P. Srinivasan, A. Leonard, J. Rogers, and R. Beyah, "Who's in Control of Your Control System? Device Fingerprinting for Cyber-Physical Systems," in *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2016.

[11] IEEE, "IEEE Standard for Electric Power Systems Communications—Distributed Network Protocol (DNP3)," 2012.

[12] P. Bosshart, D. Daly, G. Gibb, M. Izzard, N. McKeown, J. Rexford, C. Schlesinger, D. Talayco, A. Vahdat, G. Varghese, and D. Walker, "P4: Programming Protocol-Independent Packet Processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[13] S. East, J. Butts, M. Papa, and S. Shenoi, "A Taxonomy of Attacks on the DNP3 Protocol," in *Critical Infrastructure Protection III (IFIP AICT, Vol.\ 311)*. Springer, 2009, pp. 67–81.

[14] Y. Liu, P. Ning, and M. K. Reiter, "False Data Injection Attacks against State Estimation in Electric Power Grids," in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2009, pp. 21–32.

[15] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's Retransmission Timer," 2011.

[16] T. Kohno, A. Broido, and K. C. Claffy, "Remote Physical Device Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 2, no. 2, pp. 93–108, 2005.

[17] S. V. Radhakrishnan, A. S. Uluagac, and R. Beyah, "GTID: A Technique for Physical Device and Device Type Fingerprinting," *IEEE Transactions on Dependable and Secure Computing*, vol. 12, no. 5, pp. 519–532, 2015.

[18] Q. Gu, D. Formby, S. Ji, H. Cam, and R. Beyah, "Fingerprinting for Cyber-Physical System Security: Device Physics Matters Too," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 49–59, Sep. 2018.

[19] S. Jeon, J.-H. Yun, S. Choi, and W.-N. Kim, "Passive Fingerprinting of SCADA in Critical Infrastructure Network without Deep Packet Inspection," arXiv:1608.07679 [cs.CR], Aug. 2016.

[20] A. A. Cárdenas, S. Amin, Z.-S. Lin, Y.-L. Huang, C.-Y. Huang, and S. Sastry, "Attacks Against Process Control Systems: Risk Assessment, Detection, and Response," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security (ASIACCS)*, 2011, pp. 355–366.

[21] H. Lin, J. Zhuang, Y.-C. Hu, and H. Zhou, "DefRec: Establishing Physical Function Virtualization to Disrupt Reconnaissance of Power Grids' Cyber-Physical Infrastructures," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2020.

[22] H. Lin, Z. T. Kalbarczyk, and R. K. Iyer, "RAINCOAT: Randomization of Network Communication in Power Grid Cyber Infrastructure to Mislead Attackers," *IEEE Transactions on Smart Grid*, vol. 10, no. 5, pp. 4893–4906, 2019.

[23] H. Lin, B. Shrestha, and Y.-C. Hu, "Cyber-Physical Testbed: Case Study to Evaluate Anti-Reconnaissance Approaches on Power Grids' Cyber-Physical Infrastructures," in *Proc. Workshop on Learning from Authoritative Security Experiment Results (LASER)*, 2020.

[24] M. Juarez, M. Imani, M. Perry, C. Diaz, and M. Wright, "Toward an Efficient Website Fingerprinting Defense," in *Computer Security – ESORICS 2016*, 2016.

[25] A. Panchenko, F. Lanze, J. Pennekamp, T. Engel, A. Zinnen, M. Henze, and K. Wehrle, "Website Fingerprinting at Internet Scale," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2016.

[26] N. Apthorpe, D. Y. Huang, D. Reisman, A. Narayanan, and N. Feamster, "Keeping the Smart Home Private with Smart(er) IoT Traffic Shaping," *Proceedings on Privacy Enhancing Technologies (PoPETs)*, 2019.

[27] A. Mehta, M. Alzayat, R. D. Viti, B. B. Brandenburg, P. Druschel, and D. Garg, "Pacer: Comprehensive Network Side-Channel Mitigation in the Cloud," in *Proceedings of the 31st USENIX Security Symposium*, 2022.

[28] A. Sabzi, R. Vora, S. Goswami, M. Seltzer, M. Lécuyer, and A. Mehta, "NetShaper: A Differentially Private Network Side-Channel Mitigation System," in *Proceedings of the 33rd USENIX Security Symposium*, 2024.

[29] A. Sivaraman, S. Subramanian, M. Alizadeh, S. Chole, S.-T. Chuang, Agrawal, Anurag, H. Balakrishnan, T. Edsall, S. Katti, and N. McKeown, "Programmable Packet Scheduling at Line Rate," in *Proc. ACM SIGCOMM Conference*, 2016, pp. 44–57.

[30] A. G. Alcoz, A. Dietmüller, and L. Vanbever, "SP-PIFO: Approximating Push-In First-Out Behaviors using Strict-Priority Queues," in *Proc. 17th USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2020, pp. 59–76.

[31] R. Meier, P. Tsankov, V. Lenders, L. Vanbever, and M. Vechev, "NetHide: Secure and Practical Network Topology Obfuscation," in *Proc. 27th USENIX Security Symposium*, 2018, pp. 693–709.

[32] Z. Hu, H. Lin, L. Waind, Y. Qu, G. Chen, and D. Jin, "Industrial Network Protocol Security Enhancement Using Programmable Switches," in *Proceedings of the IEEE International Conference on Communications, Control, and Computing Technologies for Smart Grids (SmartGridComm)*, 2023.

[33] H. Lin, A. Slagell, C. D. Martino, Z. Kalbarczyk, and R. K. Iyer, "Adapting Bro into SCADA: Building a Specification-Based Intrusion Detection System for the DNP3 Protocol," in *Proceedings of the Eighth Annual Cyber Security and Information Intelligence Research Workshop (CSIIRW)*, 2013.

[34] I. N. Fovino, A. Carcano, T. D. L. Murel, A. Trombetta, and M. Masera, "Modbus/DNP3 State-Based Intrusion Detection System," in *2010 24th IEEE International Conference on Advanced Information Networking and Applications (AINA)*, 2010, pp. 729–736.